

Traitement Statistique avec R

Types de données en R

M. H. DIALLO

Ecole Nationale de la Statistique et de l'Analyse Economique
ENSAE Pierre Ndiaye | ISEP2

08 avril 2026

- 1 Introduction aux types de données
- 2 Objets R : environnement, class, typeof
- 3 Vecteurs atomiques
- 4 Break Session 2 "
- 5 Facteurs — Variables catégorielles
- 6 Matrices et tableaux
- 7 Listes
- 8 Data Frames et Jeux de données
- 9 Synthèse et bonnes pratiques

Section 1

Introduction aux types de données

Pourquoi les types de données sont-ils importants ?

Principe fondamental

En R, **tout est un objet** et chaque objet a un type. Connaître les types évite 80 % des erreurs de programmation.

Les types de données déterminent :

- Les **opérations possibles** sur un objet (+, mean(), paste()...)
- La **mémoire utilisée** (integer < double < character)
- Le **comportement des fonctions** (coercition implicite)
- La **structure d'une analyse** (factor character pour une régression)

À retenir

R a une **hiérarchie de types** : logical < integer < double < complex < character

Vue d'ensemble des types en R

Type	Classe	Exemple
Vecteur logique	logical	TRUE / FALSE
Vecteur entier	integer	1L, 2L
Vecteur réel	double	3.14
Vecteur texte	character	"Dakar"
Facteur	factor	factor("H","F")
Matrice	matrix	matrix(1:9, 3, 3)
Liste	list	list(a=1, b="x")
Data Frame	data.frame	data.frame(...)
Tibble	tbl_df	tibble(...)

Section 2

Objets R : environment, class, typeof

Les objets R — Concept central

Définition

Un **objet R** est tout élément stocké en mémoire avec un nom, un type interne (typeof) et une classe (class).

```
x <- 42L           # entier
y <- 3.14          # réel
z <- "Sénégal"     # texte
b <- TRUE          # logique

# Inspecter type interne vs classe
typeof(x); class(x)
```

```
## [1] "integer"
```

```
## [1] "integer"
```

```
typeof(y); class(y)
```

```
## [1] "double"
```

typeof() vs class() vs mode()

Ces trois fonctions donnent des informations **différentes** :

```
v <- 1:5           # vecteur d'entiers
typeof(v)          # type interne de stockage
```

```
## [1] "integer"
```

```
class(v)           # classe (orienté objet)
```

```
## [1] "integer"
```

```
mode(v)            # mode de stockage (héritage S3)
```

```
## [1] "numeric"
```

Règle pratique

- `typeof()` → stockage bas niveau (integer, double...)
- `class()` → comportement OO (factor, Date, lm...)
- `is.*()` → tests booléens : `is.numeric()`, `is.character()`...

L'environnement R (Environment)

```
# Créer des objets dans l'environnement global
```

```
salaire <- 450000
```

```
pays <- "Sénégal"
```

```
actif <- TRUE
```

```
# Lister les objets présents
```

```
ls()
```

```
## [1] "actif"      "b"           "pays"        "salaire" "v"           "x"
```

```
## [8] "z"
```

```
rm(list=ls())
```

```
# Voir l'environnement courant
```

```
environment()
```

```
## <environment: R_GlobalEnv>
```

Environnements imbriqués

Tester et convertir les types

```
# Tests de type
```

```
is.numeric(3.14)
```

```
## [1] TRUE
```

```
is.character("Dakar")
```

```
## [1] TRUE
```

```
is.logical(FALSE)
```

```
## [1] TRUE
```

```
# Conversion (coercition explicite)
```

```
as.integer(3.7)      # troncature → 3
```

```
## [1] 3
```

```
as.character(TRUE)   # → "TRUE"
```

```
## [1] "TRUE"
```

Section 3

Vecteurs atomiques

Qu'est-ce qu'un vecteur atomique ?

Définition

Un **vecteur atomique** est la structure de base de R. Il contient des éléments **tous du même type**. *Tout scalaire est en réalité un vecteur de longueur 1.*

Les 4 types atomiques principaux :

Type	Création	NA associé	Mémoire
logical	TRUE/FALSE	NA	4 octets
integer	1L, 2L	NA_integer_	4 octets
double	1.5	NA_real_	8 octets
character	"texte"	NA_character_	variable

Créer et manipuler des vecteurs

```
# Vecteur logique
```

```
logique2 <- c(TRUE, FALSE, TRUE, NA, "NAs")  
is.logical(logique2)
```

```
## [1] FALSE
```

```
logique3 <- as.logical(logique2)  
is.logical(logique3)
```

```
## [1] TRUE
```

```
logique <- c(TRUE, FALSE, TRUE, NA, "1")  
logique1 <- as.logical(logique)  
is.logical(logique1)
```

```
## [1] TRUE
```

```
logique4 <- c(TRUE, FALSE, TRUE, FALSE, 1)  
logique5 <- c(1, 0, 1, 0, 1)
```

```
logique5[1] <- as.logical(logique5[1])
```

Section 4

Break Session 2 ”

Break Session 2 "

##Homework deja donne ! ## Exposer texy. I; ## Opérations vectorisées

```
revenus <- c(150000, 320000, 480000, 95000, 275000)
length(revenus)
```

```
## [1] 5
```

```
# Opérations vectorisées (appliquées à chaque élément)
revenus / 1000 # diviser tous les éléments
```

```
## [1] 150 320 480 95 275
```

```
revenus
```

```
## [1] 150000 320000 480000 95000 275000
```

```
print(revenus)
```

```
## [1] 150000 320000 480000 95000 275000
```

Indexation des vecteurs

```
pays_cedeao <- c("Sénégal", "Mali", "Niger",  
                 "Burkina", "Côte d'Ivoire", "Ghana")
```

Par position

```
pays_cedeao[1]           # premier élément
```

```
## [1] "Sénégal"
```

```
pays_cedeao[c(1,3,5)]    # éléments 1, 3 et 5
```

```
## [1] "Sénégal"           "Niger"               "Côte d'Ivoire"
```

```
pays_cedeao[1:2]
```

```
## [1] "Sénégal" "Mali"
```

Par exclusion

```
pays_cedeao[-2]          # tout sauf le 2e
```

```
## [1] "Sénégal"
```

```
"Niger"
```

```
"Burkina"
```

```
"Côte d'Ivoire"
```

Valeurs manquantes (NA)

```
ages <- c(25, NA, 31, NA, 42, 29)
```

```
# Détecter les NA
```

```
is.na(ages)
```

```
## [1] FALSE TRUE FALSE TRUE FALSE FALSE
```

```
sum(is.na(ages)) # nombre de NA
```

```
## [1] 2
```

```
# Calculs en présence de NA
```

```
mean(ages) # NA si NA présents
```

```
## [1] NA
```

```
mean(ages, na.rm=TRUE) # ignorer les NA → résultat valide
```

```
## [1] 31.75
```

Section 5

Facteurs — Variables catégorielles

Qu'est-ce qu'un facteur ?

Définition

Un **facteur** (factor) est un vecteur d'entiers avec des **étiquettes** (*levels*). Il représente des **variables catégorielles** — essentiels pour les modèles statistiques en R.

Factor non ordonné

- Sexe : H / F
- Région, ethnie, secteur
- Modalités **sans ordre naturel**

Factor ordonné

- Niveau d'études
- Faible < Moyen < Élevé
- Modalités **ordonnées**

Créer et inspecter un facteur

```
# Facteur simple (non ordonné)
```

```
sexe <- factor(c("H", "F", "F", "H", "H", "F", "H"))
```

```
sexe
```

```
## [1] H F F H H F H
```

```
## Levels: F H
```

```
# Inspecter le facteur
```

```
levels(sexe) # modalités
```

```
## [1] "F" "H"
```

```
nlevels(sexe) # nombre de modalités
```

```
## [1] 2
```

```
table(sexe) # fréquences absolues
```

```
## sexe
```

```
## F H
```

Facteur ordonné (ordered factor)

```
# Facteur ordonné - niveau d'instruction
```

```
niveaux <- c("Primaire", "Secondaire", "Supérieur",  
             "Primaire", "Supérieur", "Secondaire")
```

```
educ <- factor(niveaux,  
              levels = c("Primaire", "Secondaire", "Supérieur"),  
              ordered = TRUE)
```

```
educ
```

```
## [1] Primaire    Secondaire Supérieur  Primaire    Supérieur  
## Levels: Primaire < Secondaire < Supérieur
```

```
niveaux2 <- (rep(c(1,2,3),4))  
niveaux2
```

```
## [1] 1 2 3 1 2 3 1 2 3 1 2 3
```

```
educ1 <- factor(niveaux2, levels = c(1,2,3), labels = c("Primaire", "Secondaire", "Supérieur"))  
educ1
```

Facteurs — Pièges courants

```
# Piège 1 : factor  character  
f1 <- c("10", "20", "5", "30")  
sort(f1)
```

```
## [1] "10" "20" "30" "5"
```

```
f <- factor(c("10", "20", "5", "30", "HH"))  
sort(f) # tri LEXICOGRAPHIQUE (1,10,20,30,5 !)
```

```
## [1] 10 20 30 5  HH  
## Levels: 10 20 30 5 HH
```

```
sort(as.numeric(as.character(f))) # tri numérique correct
```

```
## [1] 5 10 20 30
```

```
# Piège 2 : as.numeric() sur un factor  
# MAUVAIS : retourne les codes internes (1,2,3...)  
as.numeric(f)
```

Facteurs avec forcats (tidyverse)

```
library(forcats)
```

```
regions <- factor(c("Dakar", "Thiès", "Dakar", "Ziguinchor",  
                    "Thiès", "Dakar", "Dakar", "Diourbel"))
```

```
# Réordonner par fréquence décroissante  
fct_infreq(regions)
```

```
## [1] Dakar      Thiès      Dakar      Ziguinchor Thiès  
## [8] Diourbel  
## Levels: Dakar Thiès Diourbel Ziguinchor
```

```
# Regrouper les modalités rares  
fct_lump_n(regions, n = 2, other_level = "Autres")
```

```
## [1] Dakar  Thiès  Dakar  Autres Thiès  Dakar  Dakar  Autres  
## Levels: Dakar Thiès Autres
```

Section 6

Matrices et tableaux

Créer une matrice

Définition

Une **matrice** (matrix) est un vecteur à 2 dimensions : **lignes** × **colonnes**, un **seul type** autorisé.

```
# Matrice 3x3 remplie par colonnes (par défaut)
```

```
M <- matrix(1:9, nrow = 3, ncol = 3)
```

```
M
```

```
##      [,1] [,2] [,3]
```

```
## [1,]    1    4    7
```

```
## [2,]    2    5    8
```

```
## [3,]    3    6    9
```

```
# Remplir par lignes
```

```
M_row <- matrix(1:9, nrow = 3, byrow = TRUE)
```

```
M_row
```

```
##      [,1] [,2] [,3]
```

Opérations sur les matrices

```
A <- matrix(c(2,1,3,4), nrow=2)
```

```
B <- matrix(c(1,0,0,1), nrow=2)
```

```
# Opérations élément par élément
```

```
A + B
```

```
##      [,1] [,2]
```

```
## [1,]    3    3
```

```
## [2,]    1    5
```

```
A * 2      # multiplication scalaire
```

```
##      [,1] [,2]
```

```
## [1,]    4    6
```

```
## [2,]    2    8
```

```
# Algèbre matricielle
```

```
A %*% B      # produit matriciel
```

Tableaux multidimensionnels (array)

```
# Array 3D : 3 lignes x 3 colonnes x 2 tranches
```

```
T3D <- array(1:18, dim = c(3, 3, 2))
```

```
# Accès à la première tranche
```

```
T3D[, , 1]
```

```
##      [,1] [,2] [,3]
## [1,]    1    4    7
## [2,]    2    5    8
## [3,]    3    6    9
```

```
# Accès à la deuxième tranche
```

```
T3D[, , 2]
```

```
##      [,1] [,2] [,3]
## [1,]   10   13   16
## [2,]   11   14   17
## [3,]   12   15   18
```

Nommer lignes et colonnes

```
# Matrice de résultats d'un test
```

```
scores <- matrix(c(14,16,12,18,15,13,17,11,19),  
                 nrow = 3, ncol = 3,  
                 dimnames = list(  
                   c("Étudiant_1","Étudiant_2","Étudiant_3"),  
                   c("Maths","Stats","Informatique")  
                 ))
```

```
scores
```

```
##           Maths Stats Informatique  
## Étudiant_1    14    18           17  
## Étudiant_2    16    15           11  
## Étudiant_3    12    13           19
```

```
# Accès par nom
```

```
scores["Étudiant_1", "Stats"]
```

```
## [1] 18
```

Section 7

Listes

Qu'est-ce qu'une liste ?

Définition

Une **liste** (`list`) est une structure **récursive** pouvant contenir des éléments de **types différents** et de tailles différentes.

C'est la structure la plus **flexible** de R :

Structure	Types autorisés
Vecteur	Un seul type
Matrice	Un seul type
Data Frame	Un type par colonne
Liste	N'importe quoi (y compris d'autres listes)

Les modèles R (`lm`, `glm`) et les résultats JSON sont des listes.

Créer et accéder à une liste

Liste hétérogène

```
etudiant <- list(  
  nom      = "Ousmane Diop",  
  age      = 22L,  
  notes    = c(14, 16, 12, 18, 15),  
  admis    = TRUE,  
  adresse  = list(ville = "Dakar", quartier = "Plateau")  
)
```

Trois façons d'accéder

```
etudiant$nom           # par nom avec $
```

```
## [1] "Ousmane Diop"
```

```
etudiant[["notes"]]    # par nom avec [[]]
```

```
## [1] 14 16 12 18 15
```

```
etudiant[[2]]          # par position
```

Naviguer dans une liste imbriquée

```
# Accès à la liste imbriquée
```

```
etudiant$adresse$ville
```

```
## [1] "Dakar"
```

```
etudiant[["adresse"]][["quartier"]]
```

```
## [1] "Plateau"
```

```
# Modifier un élément
```

```
etudiant$age <- 23L
```

```
# Ajouter un élément
```

```
etudiant$filiere <- "Statistique"
```

```
# Supprimer un élément
```

```
etudiant$adresse <- NULL
```

```
str(etudiant) # structure complète
```

Parcourir une liste avec lapply/sapply

```
# Liste de vecteurs de notes
```

```
promo <- list(  
  groupe_A = c(12, 15, 14, 16, 11),  
  groupe_B = c(10, 13, 17, 12, 14),  
  groupe_C = c(16, 15, 18, 14, 13)  
)
```

```
# lapply → retourne une liste
```

```
lapply(promo, mean)
```

```
## $groupe_A
```

```
## [1] 13.6
```

```
##
```

```
## $groupe_B
```

```
## [1] 13.2
```

```
##
```

```
## $groupe_C
```

Section 8

Data Frames et Jeux de données

Le Data Frame — Structure centrale de l'analyse

Définition

Un **data.frame** est un tableau rectangulaire :

- **Lignes** = observations (individus, ménages, entreprises...)
- **Colonnes** = variables (chaque colonne est un vecteur)
- Chaque colonne peut avoir un **type différent**

C'est une liste de vecteurs de même longueur — `is.list(df)` retourne TRUE.

```
menages <- data.frame(  
  id      = 1:5,  
  region  = c("Dakar", "Thiès", "Dakar", "Diourbel", "Ziguinchor"),  
  revenu  = c(450000, 280000, 620000, 195000, 310000),  
  taille  = c(6L, 4L, 7L, 5L, 3L),  
  electrice = c(TRUE, TRUE, TRUE, FALSE, TRUE)  
)
```

Inspecter un data.frame

```
# Dimensions
```

```
nrow(menages); ncol(menages); dim(menages)
```

```
## [1] 5
```

```
## [1] 5
```

```
## [1] 5 5
```

```
# Aperçu
```

```
head(menages, 3)
```

```
##   id region revenu taille electrice
```

```
## 1  1  Dakar 450000      6      TRUE
```

```
## 2  2  Thiès 280000      4      TRUE
```

```
## 3  3  Dakar 620000      7      TRUE
```

```
# Structure complète
```

```
str(menages)
```

Résumé statistique

```
summary(menages)
```

##	id	region	revenu	taille
##	Min. :1	Length:5	Min. :195000	Min. :3
##	1st Qu.:2	Class :character	1st Qu.:280000	1st Qu.:4
##	Median :3	Mode :character	Median :310000	Median :5
##	Mean :3		Mean :371000	Mean :5
##	3rd Qu.:4		3rd Qu.:450000	3rd Qu.:6
##	Max. :5		Max. :620000	Max. :7

Accès et sélection dans un data.frame

```
# Accès à une colonne
```

```
menages$revenu
```

```
## [1] 450000 280000 620000 195000 310000
```

```
menages[["region"]]
```

```
## [1] "Dakar"      "Thiès"      "Dakar"      "Diourbel"   "Ziguinchor"
```

```
# Sélection de lignes et colonnes
```

```
menages[1:3, ] # 3 premières lignes
```

```
##   id region revenu taille electrice
```

```
## 1  1  Dakar 450000      6      TRUE
```

```
## 2  2  Thiès 280000      4      TRUE
```

```
## 3  3  Dakar 620000      7      TRUE
```

```
menages[, c("region", "revenu")] # 2 colonnes
```

```
##      region revenu
```

Le Tibble — Version moderne du data.frame

```
library(tibble)

# Créer un tibble
tb <- tibble(
  id      = 1:5,
  region  = c("Dakar", "Thiès", "Dakar", "Diourbel", "Ziguinchor"),
  revenu  = c(450000, 280000, 620000, 195000, 310000)
)

tb # affichage intelligent (type, 10 lignes max)
```

```
## # A tibble: 5 x 3
##       id region      revenu
##   <int> <chr>      <dbl>
## 1     1  Dakar    450000
## 2     2  Thiès    280000
## 3     3  Dakar    620000
## 4     4  Diourbel  195000
```

Tibble vs data.frame — Différences clés

Comportement	data.frame	tibble
Partial matching [] résultat	df\$re fonctionne df ou vecteur	Erreur (sécurité) tibble toujours
Noms de colonnes	modifiés automatiquement	préservés
Affichage	tout le tableau	10 lignes + types
Conversion chaînes	stringsAsFactors	jamais automatique

Recommandation

Utiliser `tibble()` dans tout nouveau projet tidyverse. Convertir :
`as_tibble(df)` / `as.data.frame(tb)`

Section 9

Synthèse et bonnes pratiques

Synthèse — Choisir la bonne structure

Structure	Quand l'utiliser	Contrainte
<code>vector</code>	Série de valeurs homogènes	1 seul type
<code>factor</code>	Variable catégorielle	Modalités définies
<code>matrix</code>	Calcul matriciel, algèbre	1 seul type, 2D
<code>array</code>	Données multidimensionnelles	1 seul type, nD
<code>list</code>	Données hétérogènes	Aucune
<code>data.frame</code>	Tableau de données	Colonnes = même longueur
<code>tibble</code>	Analyse tidyverse	Colonnes = même longueur

Diagnostic rapide d'un objet inconnu

```
# Créer un objet "mystère"
```

```
obj <- lm(revenu ~ taille, data = menages)
```

```
# Boîte à outils du diagnostic
```

```
class(obj)           # classe de l'objet
```

```
## [1] "lm"
```

```
typeof(obj)          # type interne
```

```
## [1] "list"
```

```
length(obj)           # nombre d'éléments
```

```
## [1] 12
```

```
names(obj)             # noms des composantes
```

```
## [1] "coefficients" "residuals" "effects" "rank"
```

```
## [5] "fitted values" "assign" "or" "df re
```

À faire

- Toujours vérifier `str()` après import
- Utiliser `is.na()` avant d'analyser
- Déclarer les facteurs **avant** la modélisation
- Préférer `tibble` à `data.frame`
- Nommer les éléments de ses listes

À éviter

- `as.numeric(factor)` sans passer par `as.character()`
- Ignorer les NA sans justification
- Mélanger types dans un vecteur (coercition silencieuse)
- Utiliser T / F au lieu de TRUE / FALSE
- Partial matching (`df$rev` au lieu de `df$revenu`)

Exercices pratiques

Exercice 1 — Vecteurs

Créer un vecteur `salaires` de 8 valeurs. Calculer la moyenne, la médiane, l'écart-type. Remplacer les valeurs $> 500\,000$ par NA.

Exercice 2 — Facteurs

Créer un vecteur `niveau_vie` avec les modalités "Pauvre", "Moyen", "Aisé" (10 observations, ordonné). Calculer les fréquences et comparer deux individus.

Exercice 3 — Data Frame

Créer un `data.frame` de 6 ménages avec 4 variables. Convertir en `tibble`. Filtrer les ménages avec `revenu > 300000`.

- Wickham, H. *Advanced R*, 2e éd. — adv-r.hadley.nz
- Wickham & Grolemund. *R for Data Science*, 2e éd. — r4ds.hadley.nz
- Documentation officielle R — cran.r-project.org
- Lafaye de Micheaux et al. (2014). *Le logiciel R*, Springer.

Questions ?

hadyafia@gmail.com